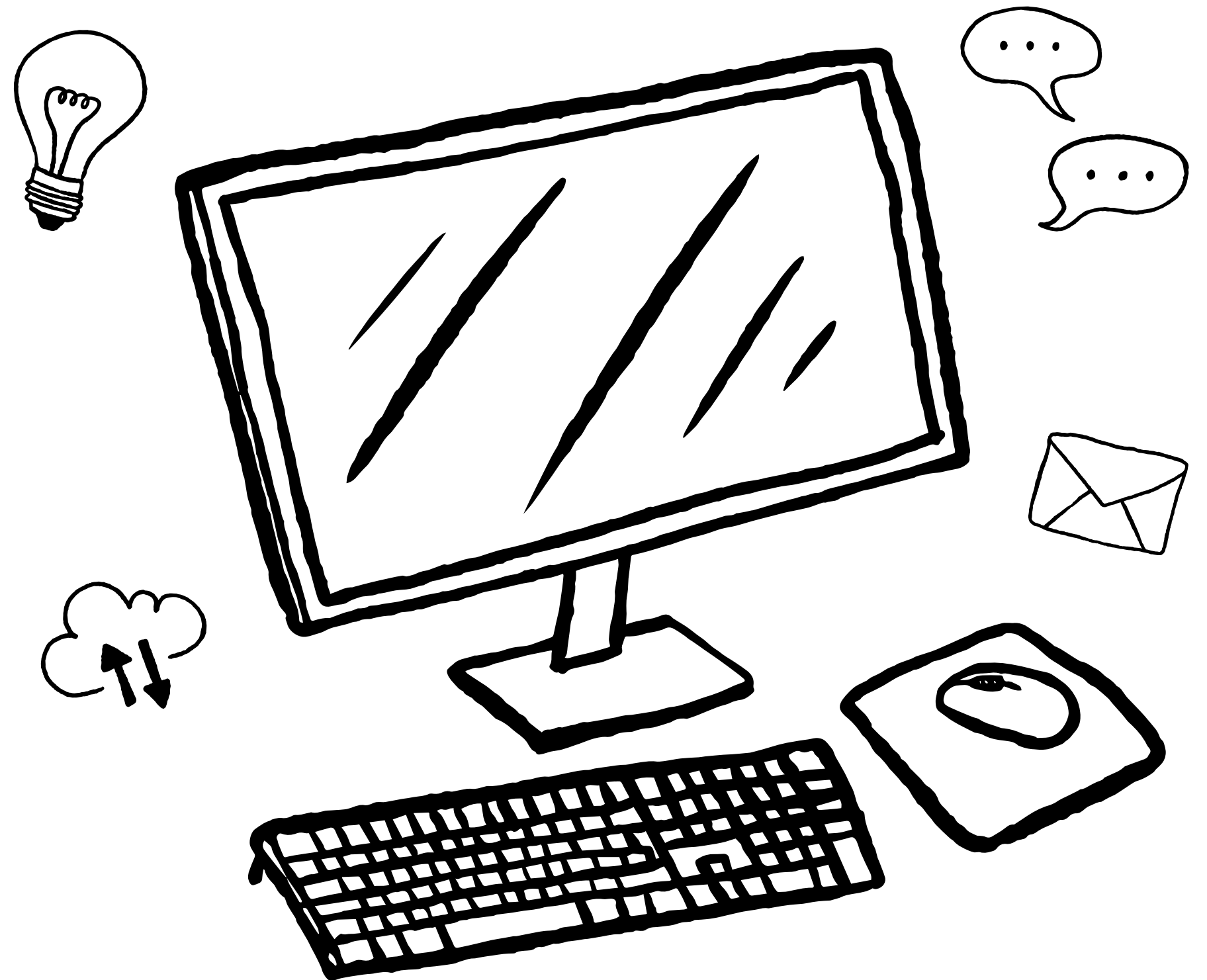


CONSUMO ENERGÉTICO, TEMPO DE RESPOSTA E POTÊNCIA

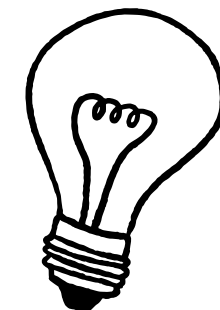
Experimento feito com métodos de ordenação

Ylanna Lima e Fernando de Oliveira





INTRODUÇÃO



Embora a notação Big-O estime o comportamento dos algoritmos com o crescimento dos dados, ela ignora efeitos do hardware real (como memória RAM, cache e sistema operacional). Este estudo analisa na prática o impacto da variação do volume de dados em diferentes algoritmos, utilizando um hardware legado otimizado por uma distribuição Linux minimalista.



AMBIENTE E METODOLOGIA

01

Configuração do Computador

Processador i3-3220 e 8GB de RAM (DDR3). Como sistema operacional: Artix (LINUX);

02

Coleta de Métricas

sudo perf stat -a -e power/energy-pkg/,duration_time,user_time,system_time

```
fanta@artix-btw /home/fanta <system>
$ stfetch

      '
      'o'
      'ooo'
      'ooxoo'
      'ooxxxoo'
      'oookkxxoo'
      'oiioxkkxxoo'
      ';;;iiioxxxoo'
      '.,;:ioxxxoo'
      '.,;:jiooo'
      'oooio-.. 'i:io'
      'ooooxxxxxoio:,, '-;'
      'ooooxxxxxkkxoooIi:-. '
      'ooooxxxxxkkkxoiiiiiji'
      'ooooxxxxxkxoiiii:' '.i'
      'ooxxxxxoi:::' '.;ioxo'
      'ooxooi:::' '.:iiixkxoo'
      'oi:' '.'';ioxxo'
      '':io'

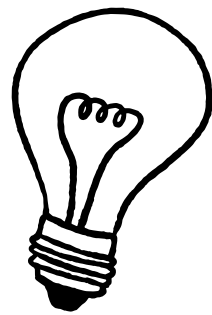
----- Hardware Information -----
🔧→ Intel(R) Core(TM) i3-3220 (4) @ 3.30 GHz
💻→ AMD Radeon RX 550 Series [Discrete]
💾→ 20.51 GiB / 108.49 GiB (19%) - ext4
📊→ 2.49 GiB / 7.70 GiB (32%)

----- Software Information -----
👤→ fanta@artix-btw
🔥→ Artix Linux x86_64
⚙️→ Linux 7.2.2-artix1-1.1
🏠→ KDE Plasma 6.7.5
🖥️→ KWin (Wayland)
🐟→ fish 4.9.3
📡→ Bluetooth 4.40.0

Performance counter stats for 'system wide':

          3,91 Joules power/energy-pkg/
234.815.639      duration_time
250.000.000      user_time
              0      system_time

0,234858617 seconds time elapsed
```



IMPLEMENTAÇÃO DOS ALGORITMOS



Bubble Sort

algoritmo de ordenação básico que percorre uma lista repetidamente, comparando elementos lado a lado e trocando-os de lugar se estiverem na ordem errada

Selection Sort

funciona procurando repetidamente o menor elemento da parte não ordenada e movendo-o para o começo.

Insertion Sort

constrói a lista ordenada um elemento por vez, pegando cada novo número e inserindo-o na posição correta entre os elementos que já foram ordenados.

BUBBLE SORT

Qual a diferença entre os códigos?

Bubble Sort V1 - o ciclo interno executa sempre até $j < n - 1$;

Bubble Sort V2 - o ciclo interno executa até $j < n - 1 - i$

```
for (int i = 0; i < n - 1; i++)
{
    for (int j = 0; j < n - 1 - i; j++)
    {
        if (v[j] > v[j + 1])
        {
            int temp = v[j];
            v[j] = v[j + 1];
            v[j + 1] = temp;
        }
    }
}
```

bubble sort v2

```

#include <iostream>
#include <queue>
using namespace std;

int main()
{
    int n;
    cin >> n;

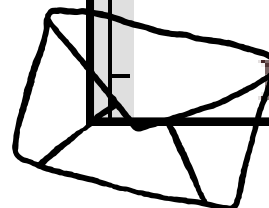
    int v[n];
    int x;

    priority_queue<int> selecao;

    for (int i = 0; i < n; i++)
    {
        cin >> x;
        selecao.push(-x);
    }

    for (int i = 0; i < n; i++)
    {
        v[i] = -selecao.top();
        selecao.pop();
    }
}

```



selection sort v2

```

for (int i=0; i < n; i++)
{
    int menorValor = i;

    for (int j = i + 1; j < n; j++)
    {
        if (v[j] < v[menorValor])
            menorValor = j;
    }

    int temp = v[i];
    v[i] = v[menorValor];
    v[menorValor] = temp;
}

```

SELECTION SORT

Qual a diferença entre os códigos?

Selection Sort V1 - tradicional do Selection Sort, utiliza dois ciclos encadeados para encontrar iterativamente o menor elemento no restante do vetor e trocá-lo com a posição atual;

Selection Sort V2 - fila de prioridade e uma simulação de uma min-heap.

INSERTION SORT

Qual a diferença entre os códigos?

Insertion Sort V1 - Realiza trocas diretas repetidas entre elementos vizinhos (**$v[j]$** e **$v[j-1]$**) para deslocar o valor passo a passo para a esquerda

Insertion Sort V2 - Guarda o valor a inserir numa variável temporária (***temp***) e apenas desloca os elementos maiores para a direita (**$v[j+1] = v[j]$**). Quando encontra a posição correta, faz uma única atribuição no final do ciclo interno (**$v[j+1] = temp$**)

insertion sort v2

```
int temp, i, j;
for ( i = 1; i < n; i++)
{
    temp = v[i];

    for( j = i - 1; j >= 0 && temp < v[j]; j--)
        v[j + 1] = v[j];

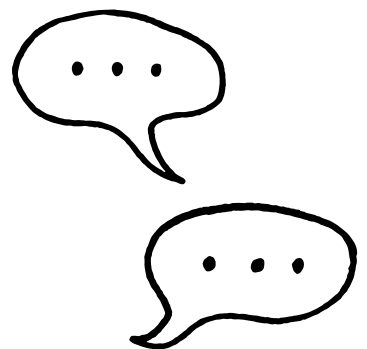
    v[j + 1] = temp;
}
```

```
int temp, i, j;
for (int i = 1; i < n; i++)

    for (int j = i; j > 0 && v[j] < v[j - 1]; j--)
    {
        int temp = v[j];
        v[j] = v[j - 1];
        v[j - 1] = temp;
    }
```

insertion sort v1

GERAÇÃO DAS ENTRADAS E AUTOMAÇÃO



Geração da Entrada

01

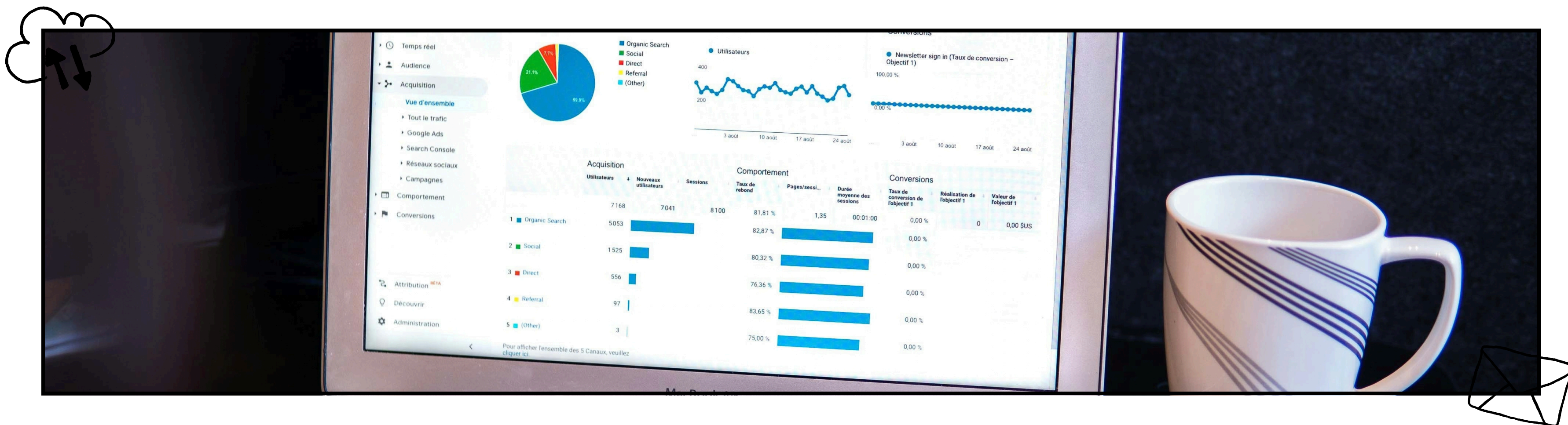
Código liberado pelo professor.

Automação

02

Para ajudar no processo dos testes, foi pensado em um código que rodasse as dez vezes o comando de métricas em cada valor. Foi escolhido um script em bash por ser mais prático na hora de rodar.

```
1  #!/bin/bash
2
3  # rodas as 10 vezes
4  for i in {1..10}
5  do
6      echo "====="
7      echo "Execução $i"
8      echo "====="
9
10     # apenas para separar as execucoes
11
12     sudo perf stat -a \
13         -e power/energy-pkg/,duration_time,user_time,system_time \
14         ./bubblesort < entradaDezMil.in \
15         2> "resultado_$i.txt"
16     # comando para pegar o resultado do perf e transformar em um arquivo txt
17 done
```



RESULTADOS OBTIDOS

COMPARATIVO ENTRE OS MÉTODOS

método bolha

V1

N	ENERGIA (J)	TEMPO DE EXECUÇÃO (MS)	TEMPO DE CPU (MS)	POTENCIA EXE	POTENCIACPU
10000	5,807	351,9	372	0,0165	0,0157
50000	155,268	9.346,70	9.937	0,0166	0,0156
100000	627,578	37.700	39.786	0,0166	0,0158

V2

N	ENERGIA (J)	TEMPO DE EXECUÇÃO (MS)	TEMPO DE CPU (MS)	POTENCIA EXE	POTENCIACPU
10000	4,16	244,3	267	0,017	0,0158
50000	112,755	6.740,80	7.122	0,0167	0,0158
100000	460,63	27.362,20	28.847	0,0168	0,016

COMPARATIVO ENTRE OS MÉTODOS

método inserção

V1

N	ENERGIA (J)	TEMPO DE EXECUÇÃO (MS)	TEMPO DE CPU (MS)	POTENCIA EXE	POTENCIACPU
10000	1,719	105,2	124	0,0163	0,0148
50000	41,862	2.544,20	2.711	0,0165	0,0154
100000	166,741	10.135,40	10.708	0,0165	0,0156

V2

N	ENERGIA (J)	TEMPO DE EXECUÇÃO (MS)	TEMPO DE CPU (MS)	POTENCIA EXE	POTENCIACPU
10000	1,021	63,6	65	0,0161	0,0159
50000	24,693	1.507,50	1.625	0,0164	0,0152
100000	97,949	5.980	6.354	0,0164	0,0154

COMPARATIVO ENTRE OS MÉTODOS

método seleção

V1

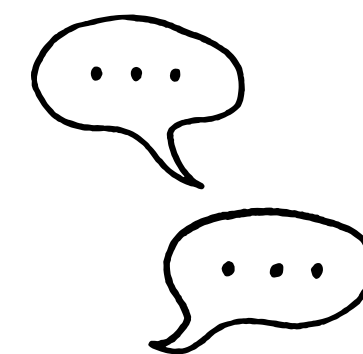
N	ENERGIA (J)	TEMPO DE EXECUÇÃO (MS)	TEMPO DE CPU (MS)	POTENCIA EXE	POTENCIACPU
10000	1,658	106,1	111	0,0156	0,0151
50000	39,675	2.457,10	2.612,00	0,0161	0,0152
100000	157,669	9.772,70	10.334,00	0,0161	0,0153

V2

N	ENERGIA (J)	TEMPO DE EXECUÇÃO (MS)	TEMPO DE CPU (MS)	POTENCIA EXE	POTENCIACPU
10000	0,169	10,7	14	0,0161	0,0135
50000	0,9	49,4	72	0,0182	0,0132
100000	1,779	98,1	134,6	0,0181	0,0135



CONSIDERAÇÕES FINAIS



O estudo demonstra que otimizar o código reduz drasticamente o consumo de energia do processador. Comprova-se que diminuir o tempo de execução é mais relevante para a economia de energia do que manter uma potência instantânea baixa. Em larga escala, essas otimizações reduzem significativamente a demanda elétrica global, tornando a sustentabilidade energética um critério essencial no desenvolvimento de software.